

Architectural Evolution of nopCommerce: Federated Commerce After Acquisitions

Part 1 - Architecture Checkpoint

Ricardo Dias [108598], Sérgio Correia [108647]

v2026-05-04

Abstract

nopCommerce is a production-grade ASP.NET Core eCommerce platform built as a modular monolith with onion architecture. This report addresses Scenario A of the assignment: federated commerce after acquisitions, where Northstar Living Group must consolidate two acquired business units under one digital foundation while preserving each unit's local autonomy. The current architecture is analysed for seams and pressure points, the domain is modeled into bounded contexts, and five quality attribute scenarios are derived to drive the design. Attribute-Driven Design (ADD) is then applied to evolve the platform across five focused iterations, producing a target architecture with eight independently deployable subsystems, five Architecture Decision Records (each with rejected alternatives), a risk register with nine scored entries, an evolution roadmap with explicit coexistence rules, and a runnable feasibility spike that proves the highest-risk decision is viable on the existing codebase. The result is a focused, traceable architecture in which every decision can be tied back to a quality attribute scenario and a concrete pressure point in the current code.

Contents

1	Introduction	3
1.1	Scenario Choice	3
1.2	Why This Creates Real Architectural Pressure	3
1.3	Why nopCommerce Makes This Interesting	4
1.4	Document Structure	4
2	Current-State Analysis of nopCommerce	5
2.1	Architecture Overview	5
2.2	Multi-Store Model	6
2.3	Architectural Seams (Extension Points)	6
2.4	Pressure Points for Scenario A	7
3	Domain and Boundary Model	8
3.1	Subdomains	8
3.2	Bounded Contexts	8
3.3	Context Map	9
3.4	Ownership of Major Responsibilities	10
3.5	Where Each Bounded Context Lives	10
4	Quality Attribute Scenarios	11
5	Chosen Framework: ADD	13
5.1	Why ADD Fits This Case	13
5.2	Why Not ACDM or ADM/TOGAF	13
6	Target Architecture	15
6.1	Consolidated Component View	16
6.2	Consolidated Deployment View	18
6.3	Cross-Cutting Concerns	18
6.4	How This Satisfies the Required Technical Shape	19
6.5	Runtime Interactions	19
7	Architectural Decisions	22
7.1	ADR-001: Per-BU process and database isolation	22
7.2	ADR-002: Federate identity via Keycloak as external IdP	23
7.3	ADR-003: Bridge IEventPublisher to RabbitMQ for cross-BU events	24
7.4	ADR-004: BU-local ERP behind anti-corruption layer with circuit breaker	25
7.5	ADR-005: Federated Meilisearch with DB fallback strategy	26
8	Risk and Validation Plan	28
8.1	Risk Register	28
8.2	Risk Heatmap	28
8.3	Mitigations	30
9	Evolution Roadmap	31

1. Introduction

Modern eCommerce platforms rarely live alone. As businesses grow through acquisition, a single platform becomes the digital foundation of multiple, formerly independent companies, each with its own catalog logic, pricing habits, inventory assumptions, and operational rhythm. The architectural challenge is not feature work; it is to evolve a known platform under genuine architectural pressure without either fragmenting it into disconnected silos or collapsing every business unit into one rigid model.

This report is part of the *Software Architecture* course and corresponds to Part 1 of the group assignment. The selected scenario is Scenario A: federated commerce after acquisitions. The base system is [nopCommerce](#), an open-source ASP.NET Core platform with a modular monolith and an onion-style layered structure.

The objective is not to rewrite nopCommerce, nor to redecorate it with fashionable technology. It is to decide which seams to enforce, which subsystems to extract, and which decisions to defend, and to do so with explicit traceability from business drivers, through quality attribute scenarios, into Architecture Decision Records and a runnable feasibility spike on the riskiest decision.

1.1 Scenario Choice

Northstar Living Group (NLG) is a retail holding company that grew through acquisition. Two of its specialist brands are modeled explicitly in this report:

- **BU1 - HomeStyle** (home decor and furniture, premium pricing, B2C);
- **BU2 - WorkSpace** (office supplies and equipment, B2B pricing, wholesale roles, volume discounts).

The brands were built independently and reflect that. Leadership wants a shared digital foundation to reduce platform cost and enable group-level insights, but cannot destroy the local autonomy that makes each brand work. Forcing a single rigid model would either homogenise what should not be homogenised or introduce so much exception logic that the “shared” platform becomes more fragile than what it replaced. Getting this wrong means either lost brand differentiation (and the customer loyalty each acquisition brought with it) or recurring outages and slow delivery that erode the cost savings the consolidation was meant to produce.

1.2 Why This Creates Real Architectural Pressure

The tension is not simply “how do we share code”. It is a structural conflict between two legitimate requirements:

Table 1: Competing forces in the federated scenario

Pull toward centralisation	Pull toward isolation
Group-level identity, single sign-on	BU-specific pricing and catalog rules
One shared customer base across brands	BU-specific roles (VIP, Trade, Wholesale)
Lower operational cost (one platform)	Independent deployability per BU
Group-level analytics and reporting	No cross-BU failure propagation

There is no neutral position: every choice about what to share and what to isolate has consequences for coupling, failure propagation, and BU autonomy. The strategic goals from the scenario brief are: (1) create a shared digital foundation across the group, (2) preserve meaningful local autonomy for each business unit, and (3) allow the platform to evolve without cross-unit instability and uncontrolled coupling.

1.3 Why nopCommerce Makes This Interesting

nopCommerce is a *store-aware but not store-enforced* system. It has a multi-store concept with per-store URLs, themes, and settings, but the isolation is opt-in. By default, products appear in all stores, customer identities are global, roles have no store scope, and inventory is warehouse-centric without BU assignment. The system already has the concept of stores; it lacks the enforcement, the boundary contracts, and the asynchronous coordination required for real federation. The work of this report is to decide what to enforce, what to extract, and what to leave shared, with explicit architectural justification.

1.4 Document Structure

This report is organized as follows. Section 2 analyses the current state of nopCommerce and identifies the architectural seams and pressure points relevant to the scenario. Section 3 models the relevant subdomains and bounded contexts. Section 4 presents five quality attribute scenarios. Section 5 justifies the choice of Attribute-Driven Design as the evolution framework. Section 6 details the target architecture in five focused ADD iterations and presents the consolidated component view, deployment view, cross-cutting concerns, and runtime interaction diagrams (Subsection 6.5) for the four flows that exercise the architecture. Section 7 collects the architectural decisions in five Architecture Decision Records, each with rejected alternatives. Section 8 documents the risk register. Section 9 outlines the evolution roadmap.

2. Current-State Analysis of nopCommerce

This section analyses how nopCommerce currently supports and conflicts with the federated commerce scenario. It identifies the architectural seams that can be reused for evolution and the pressure points where the existing design fights the federation requirements.

2.1 Architecture Overview

nopCommerce follows an **onion architecture** built on C# and ASP.NET Core. Dependencies only point inward; outer layers may reference any inner layer directly, not just the adjacent one. [Figure 1](#) shows the upstream layer diagram.

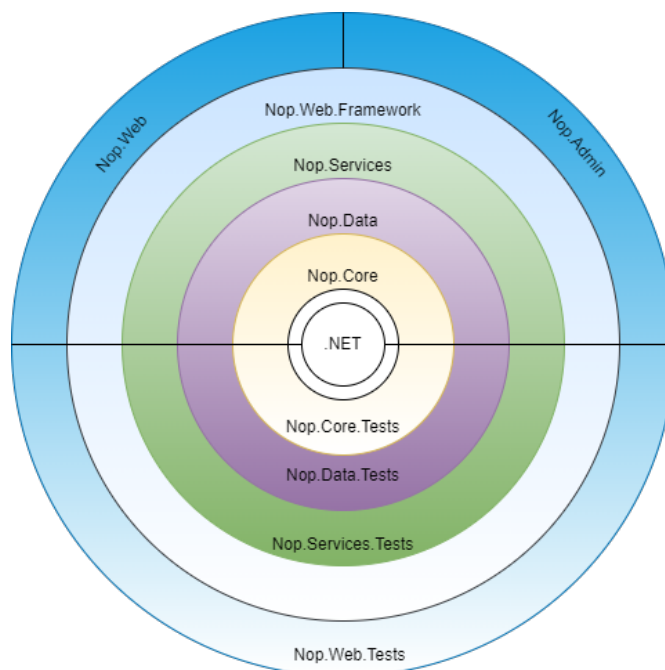


Figure 1: nopCommerce onion architecture.

Dependency rules are enforced through project references in the `.csproj` files; the projects under `src/` are listed in [Table 2](#).

Table 2: nopCommerce projects under `src/` (Nop. prefix omitted).

Project	Responsibility
Core	Domain entities, caching abstractions, and event contracts.
Data	Data access via linq2db and FluentMigrator.
Services	Business logic: orders, tax, shipping, catalog, scheduled tasks.
Web.Framework	Presentation infrastructure: validation, bundling, middleware pipeline.
Web	ASP.NET Core MVC host: public storefront and admin area.
Plugins	Extensions with their own data access, controllers, and views.

The platform ships as a single deployable unit: one ASP.NET Core process backed by a single relational database, with no hard service boundaries between layers. There is no built-in mechanism for running parts of the system as independent services or for isolating one store's failure from another's; a memory leak, a slow plugin, or a runaway query in BU1 would degrade BU2 in the same instance. This is the structural reason the target architecture runs each BU as its own nopCommerce process rather than relying on the existing multi-store feature alone.

Within the current monolith, plugins share the DI container, so nothing prevents one plugin from taking a direct dependency on another's services. The convention is to communicate through in-process domain events, but this is enforced by review, not by the architecture. The risk is that direct references can creep in, and once they do, extracting any single plugin into an out-of-process service becomes a non-trivial refactor: the consumer of those direct calls must be migrated to events at the same time.

2.2 Multi-Store Model

nopCommerce has a built-in multi-store concept. Each store has its own URL, company metadata, theme, and settings. However, the isolation is **opt-in**: products, categories, and manufacturers use a generic bridge table that is only applied when an entity is explicitly flagged as store-limited. Without explicit mappings, all products appear in all stores by default. Settings can be overridden per store, with a global fallback when no override exists. There is no concept of a *store group*, store hierarchy, or parent-child store relationship.

2.3 Architectural Seams (Extension Points)

The existing codebase exposes two kinds of seams that the target architecture leans on rather than reinvents.

The first kind is **store-scoping mechanisms** that apply the multi-store boundary at the data layer. Per-store settings have a global fallback when no override exists. Orders are stamped with their store at creation and filtered everywhere downstream. Tier pricing supports per-store rules but is opt-in per product. Catalog isolation uses a generic store-mapping bridge, also opt-in today. Most of these are solid; tier pricing and the catalog mapping are partial because they require explicit configuration on every entity that should be isolated.

The second kind is **extension contracts**: the plug-in slots that allow new behaviour to be added without modifying the core. The external-authentication contract is the OIDC integration point already used by the Facebook plugin upstream. The in-process domain-event publisher, with its consumer interface and automatic per-entity events fired from the repository layer, is the publish surface for domain events; the abstraction is solid, but the transport is purely in-process.

2.4 Pressure Points for Scenario A

The following architectural conflicts in the codebase directly affect the federated scenario.

P1 - Product base price is global

The product price is a single decimal field with no store dimension. Per-store pricing requires the tier-price mechanism, which is opt-in complexity that must be explicitly configured per product per store. BUs cannot independently own their pricing logic without either maintaining a parallel tier price for every product or introducing an external pricing service.

P2 - Customer identity is fully global

Customer email and username are global, with no database-level uniqueness constraint and no per-store uniqueness in the application-level checks. `RegisteredInStoreId` is set at registration time but is never used for access control: it appears only in incomplete-registration follow-up filters and notification-email routing. Customer roles have no store dimension. BUs cannot have private customer bases or BU-specific roles. A customer registered in BU1 has implicit access to BU2.

P3 - Inventory is warehouse-centric, not BU-aware

Warehouses are not store-aware. There is no store dimension on either the warehouse entity or the link table that connects products to warehouses. All stores draw from the same warehouse pool. BUs cannot maintain independent inventory assumptions or stock allocation. A stock depletion in BU1 is a stock depletion everywhere.

P4 - No cross-process coordination mechanism

`nopCommerce` has an in-process event system with publisher and consumer abstractions and automatic create, update, delete events fired from the repository layer on every CRUD operation. The system is in-process only, sequential (consumers are awaited one after another), and non-durable (errors are logged but not retried or persisted). Any federated behaviour across BUs requires external messaging infrastructure. The existing event abstraction can serve as a publish point, but the transport must be replaced or augmented with a durable broker.

P5 - Settings eventual consistency

When global settings are updated, all stores are affected immediately with no coordination or approval step. A group-level admin change can affect all BUs simultaneously with no isolation.

P6 - IgnoreStoreLimitations is a global bypass

When this flag is on, the store-mapping filter short-circuits immediately and returns the unfiltered query regardless of any per-entity isolation flags or mapping records. All catalog isolation is silently disabled. A single admin toggle disables BU isolation across the group with no warning.

P7 - ACL is role-scoped only, not store-scoped

The ACL mechanism restricts product and category visibility by customer role only; there is no store-aware ACL. A product visible to a wholesale role is visible in every store where that role exists. BU-specific product access control requires workarounds, not a clean mechanism.

3. Domain and Boundary Model

This section turns the scenario and the current-state pressure points into a domain model: which subdomains exist, how they group into bounded contexts, who owns what, and how contexts interact at runtime.

3.1 Subdomains

The federation scenario maps to nine subdomains across the three DDD classifications: core, supporting, and generic.

The **core** subdomains are identity and access, the cross-BU customer profile, pricing, and orders. Identity and customer profile are core to the group's value proposition (single sign-on across BUs, unified customer view) and are shared. Pricing (price lists, discounts, tier rules) and orders (order lifecycle, workflows, fulfilment) are core to each BU's competitive position, intentionally divergent per BU, and remain locally owned.

The **supporting** subdomains are catalog management (product definitions, attributes, categories), inventory (stock levels, warehouse allocation), and configuration (platform settings such as theme, tax, and shipping methods). All three are scoped per BU, with each unit owning its own.

The **generic** subdomains are notifications and messaging (emails, reminders, campaigns) and search and discovery (cross-BU product discovery). These are solved problems shared as group infrastructure.

3.2 Bounded Contexts

Five bounded contexts are derived from the subdomains. Each context has a single owner, an explicit responsibility, and a defined external boundary.

BC1 - Group Identity (shared)

Owned by the Northstar Group platform. Authenticates users across all BUs, issues OIDC tokens, and manages delegated access. Its contents are customer identity, the SSO infrastructure (Keycloak), and group-level roles (Registered, Verified). BUs consume identity from this context as relying parties; they do not own it. BU-specific roles and BU customer preferences live elsewhere.

BC2 - BU1 Commerce (HomeStyle)

Owned by the BU1 operational team. Responsible for the HomeStyle catalog, pricing, inventory, and orders. Contents are BU1 products, BU1 tier prices, BU1 warehouse stock, BU1 orders, and BU1-specific roles (VIP, Trade). Receives identity from BC1; publishes order events to BC4. Does not contain BU2 products or shared customer-profile writes.

BC3 - BU2 Commerce (WorkSpace)

Owned by the BU2 operational team. Responsible for the WorkSpace catalog, pricing, inventory, and orders. Contents are BU2 products, BU2 wholesale pricing, BU2 warehouse stock, BU2 orders, and BU2-specific roles (Wholesale, B2B Account). Receives identity from BC1; publishes order events to BC4. Does not contain BU1 products or shared customer-profile writes.

BC4 - Group Customer Profile (shared)

Owned by the group platform. Aggregates customer activity across BUs for the shared CRM view. Contents are cross-BU purchase history, communication preferences, and loyalty data. Consumes order events from BC2 and BC3; exposes a read model to CRM users. Does not contain order management or BU-specific pricing.

BC5 - Shared Infrastructure

Owned by the group platform team. Provides search indexing, the messaging bus, and notifications. Contents are the search index (Meilisearch), the message broker (RabbitMQ), and email and notification services. BUs publish to and consume from this context.

3.3 Context Map

Figure 2 shows the context map. BC1 emits OIDC tokens that BC2 and BC3 trust as relying parties. BC2 and BC3 publish order events asynchronously to BC5 through BC5 (the broker). All cross-BU coordination is mediated by BC5; BC2 and BC3 never call each other directly.

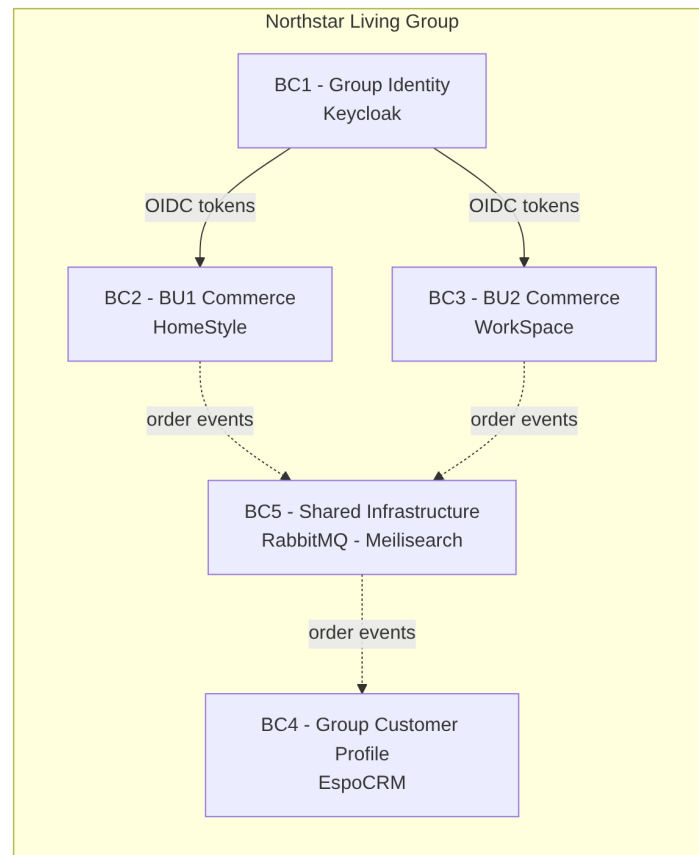


Figure 2: Bounded context map for Northstar Living Group.

3.4 Ownership of Major Responsibilities

Ownership in this design follows one rule: anything that can be owned by a single BU is owned by that BU; anything that requires a cross-BU view sits in a group context. Customer identity is the only group-owned thing the BUs depend on synchronously: BC1 (Keycloak) is the source of truth, and each BU is a relying party that consumes tokens but never owns identity itself. The full commercial domain (products, pricing, stock, and the order lifecycle from creation through fulfilment) is owned per BU, by BC2 (HomeStyle) and BC3 (WorkSpace), each running its own catalog, its own tier prices, its own warehouse and ERP, and its own order pipeline with the store dimension enforced at every step. Cross-BU state flows in one direction only, through the broker (BC5): the BUs publish order events, and BC4 (the group customer profile in EspoCRM) consumes them as a read model and never writes back. The shared infrastructure context (BC5) owns the generic capabilities that none of the BUs needs privately: the federated search index (Meilisearch), the asynchronous transport (RabbitMQ, with retry and dead-letter handling), and notifications.

3.5 Where Each Bounded Context Lives

Each bounded context maps to a concrete piece of running software. Only BC2 and BC3 stay inside the nopCommerce monolith, each running as its own nopCommerce process with its own database. BC1 is hosted by Keycloak; BC4 is hosted by EspoCRM; and the three capabilities of BC5 live in standalone services: RabbitMQ for the broker, Meilisearch for the federated search index, and a small notification service for outbound messaging. Each BU additionally runs a local ERP behind an anti-corruption layer; the ERP is part of the BU's commerce context but executes in its own process so that an ERP outage cannot take down its BU's storefront. The full deployment topology, including the eight independently deployable subsystems and their persistence, is detailed in Section 6; the rationale for each extraction is in Section 7.

4. Quality Attribute Scenarios

Five quality attribute scenarios drive the design. Each follows the standard six-part format and traces back to a pressure point and a bounded context. Together they cover availability, modifiability, security, reliability, and consistency.

QA1 - Fault Isolation (Availability) (*P3, P4; BC2 / BC3*)

- Source: BU2's local ERP.
- Stimulus: ERP crashes or becomes unreachable; all HTTP calls from BU2's nop-Commerce instance to the ERP time out.
- Artifact: BU1 storefront, group identity service, shared search index, BU2 storefront.
- Environment: Normal mid-day traffic on both storefronts.
- Response: BU1's storefront, the group identity service, and the shared search index continue without degradation. BU2 enters a degraded-but-open state with cached stock and a "stock to be confirmed" UX.
- Response Measure: BU1 p95 response time stays under 300 ms; BU2 shows cached or stale inventory within 5 s of detection; group identity and search remain fully available; the failure is visible in BU2 admin within 30 s.

QA2 - BU Pricing Autonomy (Modifiability) (*P1, P6; BC2 / BC3*)

- Source: BU1 product manager.
- Stimulus: Updates pricing rules for 200 products, including new tier prices and promotional discounts.
- Artifact: BU1 catalog and pricing; BU2 catalog and pricing.
- Environment: Both storefronts live; normal operation.
- Response: BU1's catalog and pricing reflect the change immediately for BU1 customers. BU2's catalog and pricing remain entirely unaffected, with no redeployment, no shared configuration change, and no coordination with BU2 required.
- Response Measure: The BU1 update is visible to customers within 60 s; BU2 pricing queries return unchanged values; no global setting is touched during the operation.

QA3 - Federated Identity (Security / Interoperability) (*P2; BC1*)

- Source: Customer already authenticated on BU1 via Keycloak SSO.
- Stimulus: Navigates to BU2's storefront URL.
- Artifact: BU2 storefront, authentication session, per-BU customer role assignments.
- Environment: Normal operation; Keycloak available.
- Response: BU2 recognises the customer without re-authentication, applying BU2-specific roles only inside BU2 and never exposing BU1 roles. Both BUs trust the OIDC token from Keycloak.
- Response Measure: No credential prompt on BU2 login initiation; session transition under 2 s; zero role bleed-through in integration tests.

QA4 - Search Degradation and Recovery (Reliability) (*P4; BC5*)

- Source: Meiliseach instance.
- Stimulus: Becomes temporarily unavailable through a process crash or network partition.
- Artifact: Both BU storefronts' search path; the Meiliseach index.
- Environment: Both storefronts live and serving traffic.
- Response: Both storefronts detect the failure and fall back to direct DB search within a configured timeout (slower but available). On recovery they resume Meiliseach and the index resyncs from the broker backlog.
- Response Measure: Fallback activates within 3 s of detection; fallback search returns results in under 5 s; resync completes within 5 minutes of recovery; no data is permanently lost.

QA5 - Cross-BU Order Visibility (Consistency / Observability) (*P4; BC4*)

- Source: Customer.
- Stimulus: Places and confirms an order on BU2's storefront.
- Artifact: BU2 nopCommerce instance, RabbitMQ, EspoCRM CRM consumer, the cross-BU customer profile.
- Environment: Normal operation; RabbitMQ available; CRM consumer running (or temporarily down for the degraded sub-scenario).
- Response: BU2 publishes the order event to RabbitMQ; the CRM consumer processes it and updates the cross-BU customer profile in EspoCRM. If the consumer is down, the message persists in the durable queue and is processed on recovery without loss.
- Response Measure: The CRM is updated within 30 s under normal conditions; the message survives at least 24 h of consumer downtime; the backlog is processed without duplicates by an idempotent consumer.

5. Chosen Framework: ADD

Attribute-Driven Design (ADD 3.0) is the primary software architecture design framework for this evolution. ADD is an iterative decomposition method in which architectural decisions are driven by quality attribute scenarios and architectural drivers, not by feature lists or technology preferences. Its core loop is: select an element to decompose, identify the architectural drivers, choose an architectural pattern or tactic that responds to those drivers, instantiate the pattern and allocate responsibilities, define interfaces and interactions, verify the decision against the drivers, and repeat for sub-elements.

5.1 Why ADD Fits This Case

Four things make ADD the right fit for this scenario. First, the assignment requires three to five concrete quality attribute scenarios, which is exactly the input ADD is built around. Each scenario in Section 4 maps directly to one ADD iteration in Section 6, as shown in Table 3.

Table 3: QA scenarios as ADD iteration drivers

Driver	ADD iteration	Decision driven
QA1, QA2	1	Per-BU process and database isolation
QA3	2	External IdP (Keycloak); nopCommerce as relying party
QA5	3	Bridge IEventPublisher to RabbitMQ via transactional outbox
QA1	4	BU-local ERP behind ACL with circuit breaker
QA4	5	Federated Meiliseach with DB fallback strategy

Second, ADD supports brownfield evolution. The current-state analysis in Section 2 establishes the baseline, and each iteration decides what to enforce, extract, or leave unchanged; the framework explicitly asks what is already there, what constraints exist, and what must change to satisfy the drivers, which is a better fit than frameworks that assume a blank slate.

Third, ADD has traceability built in. The grading rubric explicitly weights traceability from drivers to implementation, and ADD’s output satisfies that naturally: each ADR in Section 7 references the QA scenario, pressure point, bounded context, and ADD iteration that produced it.

Fourth, ADD’s decomposition is bounded by the selected drivers: you stop when the drivers are satisfied, not when you run out of technologies to add. That matches the brief’s warning against inflated designs that are only convincing in diagrams.

5.2 Why Not ACDM or ADM/TOGAF

ACDM (Architecture Centric Design Method) is more comprehensive: it includes explicit phases for concept generation, architectural evaluation, and iterative refinement, and

is built around producing multiple candidate architectures and choosing between them. The scenario does have competing stakeholder views (group leadership pulling toward centralisation, BU teams pulling toward autonomy), and ACDM is well suited to greenfield systems where the design space is genuinely open and concept generation has room to run. Here the design space is heavily constrained by the existing nopCommerce platform, the existing seams, and the brownfield rule that the platform must keep running, so most of ACDM's concept-generation effort would be spent rejecting alternatives that are already foreclosed by the codebase. The competing stakeholder views are captured directly through the QA scenarios in Section 4 (each one represents a concrete stakeholder pressure point), and ADD's iteration-per-driver shape asks the right brownfield question: given this existing element and this driver, what is the smallest change? The grading rubric's emphasis on traceability from drivers to ADRs also lands more cleanly on ADD's structure than on ACDM's broader process.

ADM / TOGAF is an enterprise architecture framework covering business, data, application, and technology layers across an entire organisation. It is appropriate when the scope is the full enterprise IT landscape with multiple programmes. For a focused evolution of a single commerce platform, TOGAF introduces governance overhead (architecture repository, compliance assessment, enterprise migration planning) that is disproportionate to the problem.

6. Target Architecture

The target architecture is built by applying ADD one iteration at a time. Each iteration takes a single QA driver, refines one element of the system, selects design concepts that respond to the driver, and analyses what improved and what remains open. Five iterations cover QA1–QA5; the consolidated component view, deployment view, and cross-cutting concerns are presented at the end of the section.

Iteration 1 - Per-BU Process and Database Isolation (*drivers: QA1, QA2*)

The goal is structural separation of BU failure domains so a memory leak, slow plugin, runaway query, or DB lock in one BU cannot degrade another, and so each BU's catalog and pricing live in their own database with no cross-BU coordination overhead at the data layer. The element refined is the deployment topology and the database boundary. The selected concepts are per-BU ASP.NET Core processes (no shared GC, no shared DI container, no shared connection pool), per-BU PostgreSQL databases (no shared schema, no shared migrations), and the existing multi-store feature relegated to within-BU concerns where it remains useful for sub-stores but is no longer load-bearing for cross-BU isolation.

This is recorded as ADR-001 (per-BU process and database isolation); the accepted trade-off is operational doubling, with two deployments to maintain, two database lifecycles, and “publish to every BU” workflows becoming explicit multi-step admin operations rather than a single global click. QA1 is satisfied by construction (no shared failure domain at the process or database level), QA2 follows trivially (each BU has its own `Product` table with its own `Price` column), and three pressure points dissolve structurally rather than being mitigated: P1 (global product price), P6 (the `IgnoreStoreLimitations` bypass demotes to a within-BU concern), and P7 (role-only ACL becomes a within-BU concern with no cross-BU leakage).

Iteration 2 - Federated Identity (*driver: QA3*)

The goal is to recognise a customer across BUs without re-authentication and apply BU-specific roles only inside each BU. With per-BU databases now enforced (Iteration 1), identity is the one piece of cross-BU shared state that must remain in lock-step. The element refined is the authentication seam in nopCommerce's web framework. The selected concepts are an external IdP (Keycloak) so identity becomes a shared asset with nopCommerce as a relying party, OIDC for protocol reuse on the existing extension slot, and group claims for group-wide roles with BU claims mapped to local roles per BU.

This is recorded as ADR-002 (federate identity via Keycloak as external IdP); the accepted trade-off is that login latency now depends on Keycloak availability, mitigated by short-lived tokens with refresh and a Keycloak HA pair. QA3 is satisfied (one identity, two BUs, no re-auth), but role-mapping rules between Keycloak groups and nopCommerce roles must be specified, and identity becomes a group-level SPOF that requires HA.

Iteration 3 - Cross-BU Async Coordination (*driver: QA5*)

The goal is that orders placed in any BU must reach the group customer profile within 30 seconds under normal conditions and survive a CRM consumer outage of at least 24 hours without data loss. With per-BU databases (Iteration 1), the in-process event publisher (P4) cannot reach across BUs and is in any case non-durable. The element refined is the event-publication path. The selected concepts bridge the existing publisher to RabbitMQ

to reuse the existing publish points, use a topic exchange per BU so consumers subscribe by interest, use durable queues with dead-letter and an idempotent consumer (the CRM upserts on order ID), and wrap the publisher in a transactional outbox that writes the order and the outbox row in the same DB transaction with a relay draining to RabbitMQ.

This is recorded as ADR-003 (bridge the event publisher to RabbitMQ via transactional outbox); the accepted trade-off is eventual consistency between the BU order and the group profile, bounded by QA5 (≤ 30 s normal, ≤ 24 h under degradation). QA5 is satisfied, the same broker also unlocks ADR-005 (search re-index events), and the remaining open question is bounding the backlog drain time after long outages.

Iteration 4 - Fault Isolation Across BU ERPs (*driver: QA1*)

The goal is that a BU's local ERP failure must not cascade across the group. Iteration 1 already isolates BU failure domains at the process and database level; this iteration extends the same principle to the synchronous ERP integration so that an ERP outage degrades only its own BU rather than failing the storefront. The element refined is the integration boundary between each nopCommerce instance and the BU's local ERP. The selected concepts are a per-BU ERP, an anti-corruption layer so each BU ERP keeps its own model, a circuit breaker to contain cascading failure when the ERP slows or becomes unreachable, and a cached stock snapshot with degraded-but-open mode so browse and add-to-cart continue with cached stock while checkout shows "stock to be confirmed" rather than failing.

This is recorded as ADR-004 (BU-local ERP behind anti-corruption layer with circuit breaker); the accepted trade-off is that stock numbers can be stale during an ERP outage, bounded by 5 s detection and at most 5 minutes stale before falling back to the "to be confirmed" UX. QA1 gains its second layer of fault isolation (Iteration 1 isolates the process and database; this iteration isolates the ERP integration), though the reconciliation policy for "to be confirmed" orders after recovery still needs to be defined.

Iteration 5 - Search Degradation and Recovery (*driver: QA4*)

The goal is that both storefronts continue serving search results when Meilisearch is unreachable, and resync automatically once it recovers. The element refined is the product discovery path on each BU storefront and the shared indexing pipeline. The selected concepts are an external federated search index (Meilisearch) for cross-BU discovery, a strategy pattern at the search service to switch between Meilisearch (primary) and DB search (fallback) at runtime, a health probe and circuit breaker to decide when to fall back and when to come back, and async re-indexing from RabbitMQ so on recovery the index drains pending product-changed events.

This is recorded as ADR-005 (federated Meilisearch with DB fallback strategy); the accepted trade-off is that fallback search is slower (≤ 5 s versus sub-second), and the UI shows a "results may be limited" notice during fallback. QA4 is satisfied, with the index drift bound during long outages still needing operational definition.

6.1 Consolidated Component View

Figure 3 shows the consolidated component view. Solid lines are synchronous calls; dotted lines are asynchronous events through the broker.

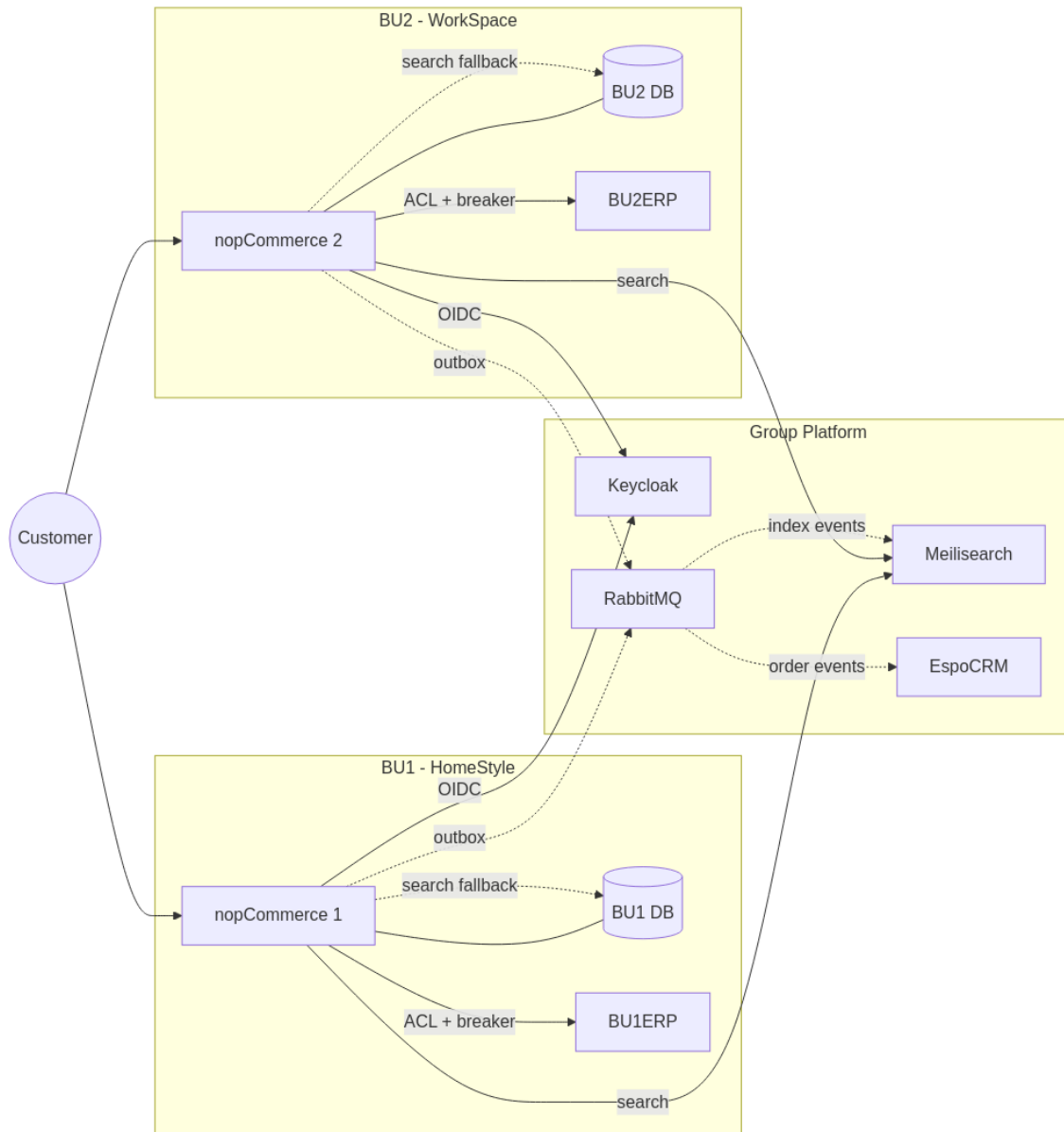


Figure 3: Consolidated component view of the target architecture. Solid lines are synchronous; dotted lines are asynchronous events through the broker.

Per-BU isolation (QA1, QA2, ADR-001) appears as the private database per BU, with no direct line between the BUs. Federated identity (QA3, ADR-002) appears as the customer's redirect to Keycloak that both BUs trust. Cross-BU coordination (QA5, ADR-003) is the dotted path from outbox to RabbitMQ to CRM, and the same path to Meilisearch for index events. Fault isolation across BU ERPs (QA1, ADR-004) shows up as the ACL plus breaker mediating each BU's ERP call. Search degradation (QA4, ADR-005) appears as the customer's primary search going to Meilisearch, with the fallback to the BU's own database when the primary fails.

6.2 Consolidated Deployment View

Each BU runs its own nopCommerce process (ASP.NET Core 9) with the OIDC and outbox plug-ins, backed by its own Postgres database. Alongside it, the BU runs its local ERP, each with its own database.

The group platform runs four standalone services: Keycloak for identity (JVM, deployed as an HA pair with its own database), Meilisearch for the federated search index (Rust, owns its own state), RabbitMQ as the event bus, and an EspoCRM consumer (PHP) for the cross-BU customer profile, also with its own database.

This gives eight independently deployable subsystems with no shared database spanning extracted boundaries. The architectural anchor for the demo is the **OIDC plus outbox plug-in pair on nopCommerce**, since that is where the federation tension is most visible.

6.3 Cross-Cutting Concerns

Some concerns are not owned by a single iteration; they constrain every iteration and run through the whole architecture.

BU scoping (*ADR-001, ADR-003, ADR-005*)

Every persisted row, event payload, and search document carries a BU and store identifier, and group-level reads aggregate across the BUs explicitly.

Identity and authorization (*ADR-002, ADR-001*)

A single Keycloak issuer is used across the group, group claims travel inside the OIDC tokens, and each BU maps those claims to local roles via a single mapping table loaded at startup and checked into git. Local role IDs are prefixed with the BU name to prevent collision across BUs.

Secrets and configuration

Connection strings, broker URIs, OIDC secrets, and ERP credentials live in environment variables; the claim-to-role mapping is config-as-code. No secret is stored in the database.

Observability (*iterations 2-4*)

Structured logs include the BU, the correlation identifier, and the customer's stable subject. Metrics cover breaker state transitions, outbox lag, search fallback rate, and consumer lag. Traces propagate through the broker.

Reliability primitives (*ADR-004, ADR-005*)

A single timeout, retry, and circuit-breaker policy is reused across the ERP integration, the broker publish path, and the search read path. The policy is defined once and configured per integration.

Consistency model (*QA5, ADR-003*)

Strong consistency inside each BU's database; eventual consistency between BUs and to group services, bounded by QA5. When the bounds slip, the slip is visible to the user through degraded banners rather than being silent.

Contract versioning (*ADR-003*)

Event payloads carry an explicit version field. Adding a field is non-breaking; renaming

or removing creates a new event type.

Failure-mode UX (ADR-004, ADR-005)

Degradation is always visible to the user: a banner, “stock to be confirmed”, or “limited results”. There are no silent fallbacks.

6.4 How This Satisfies the Required Technical Shape

The brief’s technical constraints are traced directly into the architecture. The async workflow requirement is satisfied by the order outbox path introduced in Iteration 3. Explicit reliability decisions appear twice: a circuit breaker on the BU ERP integration (Iteration 2) and a search fallback strategy (Iteration 4). The architecture surfaces six surrounding systems (Keycloak, RabbitMQ, Meiliseach, EspoCRM, and one ERP per BU), well above the two-system minimum. The independently deployable subsystem is the OIDC plug-in plus outbox relay, justified by QA1, QA3, and QA5. *No database is shared across extracted boundaries*: each BU keeps its own DB, and the ERP, identity provider, broker, search index, and CRM each hold separate state. What stays in the monolith (orders, settings, payments, shipping) is justified in Section 3.

6.5 Runtime Interactions

Flow 1 - SSO login on BU1 (QA3, ADR-002)

A guest customer authenticates on BU1’s storefront via Keycloak. nopCommerce is the relying party; Keycloak owns the credentials. After successful login, the BU1 instance creates or matches a local Customer row keyed by the Keycloak sub claim and applies the BU1-local roles derived from the token’s group claims. Note that `response_mode=query` is required: the default `form_post` triggers a SameSite cookie failure because the correlation cookie is not sent on cross-site POSTs (spike report, issue 10).

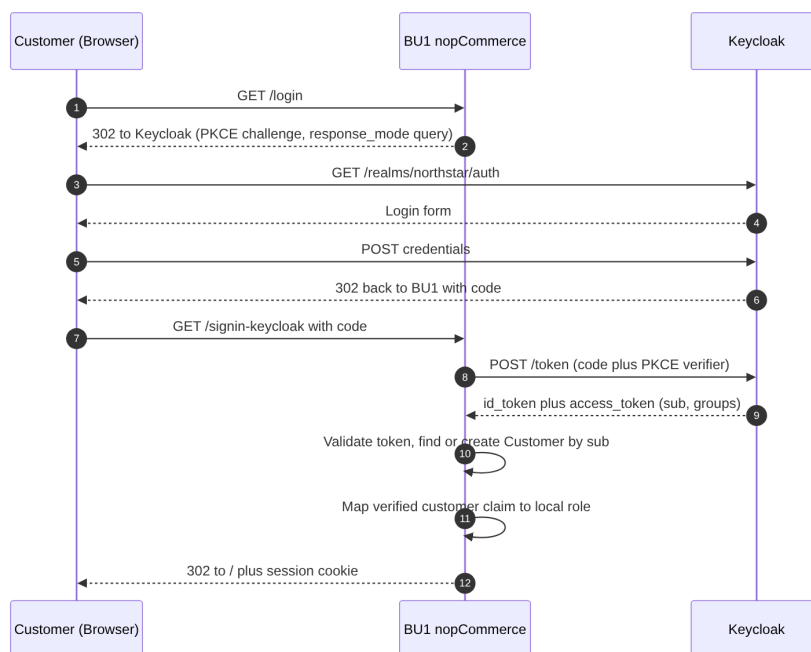


Figure 4: Flow 1: Authorisation-code flow on BU1.

Flow 2 - Cross-BU recognition (QA3, ADR-002)

A customer already authenticated on BU1 navigates to BU2 in the same browser session. The browser sends the existing Keycloak session cookie when BU2 redirects to /auth; Keycloak recognises the session and issues a code without prompting. BU2 creates a separate local Customer record (its own DB) but linked by the same sub claim. Two Customer rows exist after this flow, one in each BU's database, with the same ExternalIdentifier but different Id and different role mappings; this is intentional (BU autonomy).

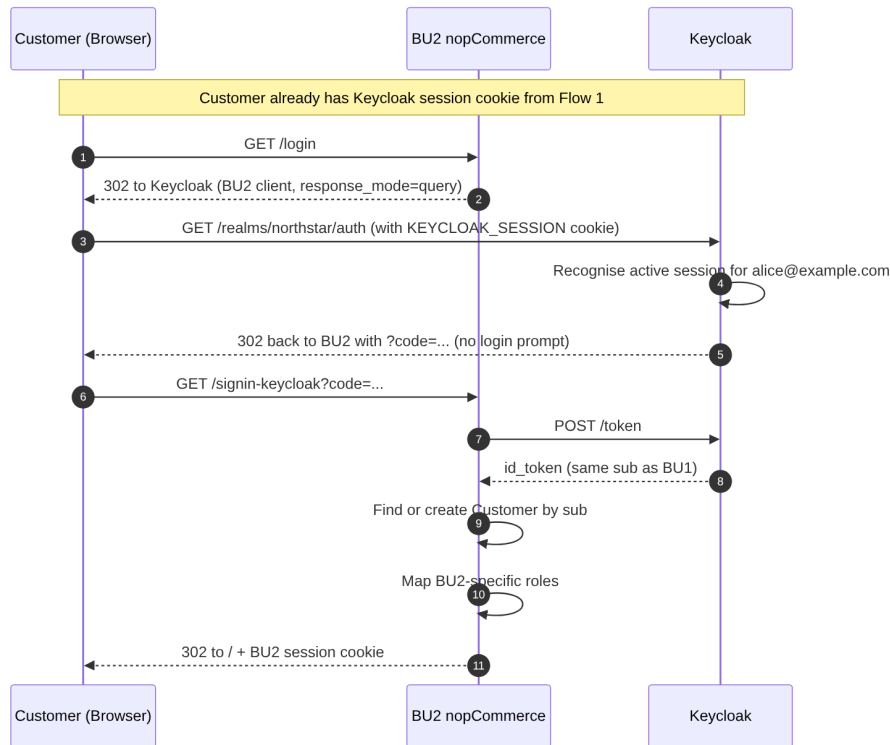


Figure 5: Flow 2: Silent SSO on BU2 reusing the Keycloak session.

Flow 3 - Order placement, outbox to broker to CRM (QA5, ADR-003)

A customer places an order on BU2. The order row and the outbox row are written in the same database transaction. A relay process publishes outbox rows to RabbitMQ and marks them as published. The CRM consumer ingests the event and idempotently updates the cross-BU customer profile. If the relay crashes between RabbitMQ ack and the outbox published_at write, the message is re-published on restart and the consumer deduplicates by (OrderId, version).

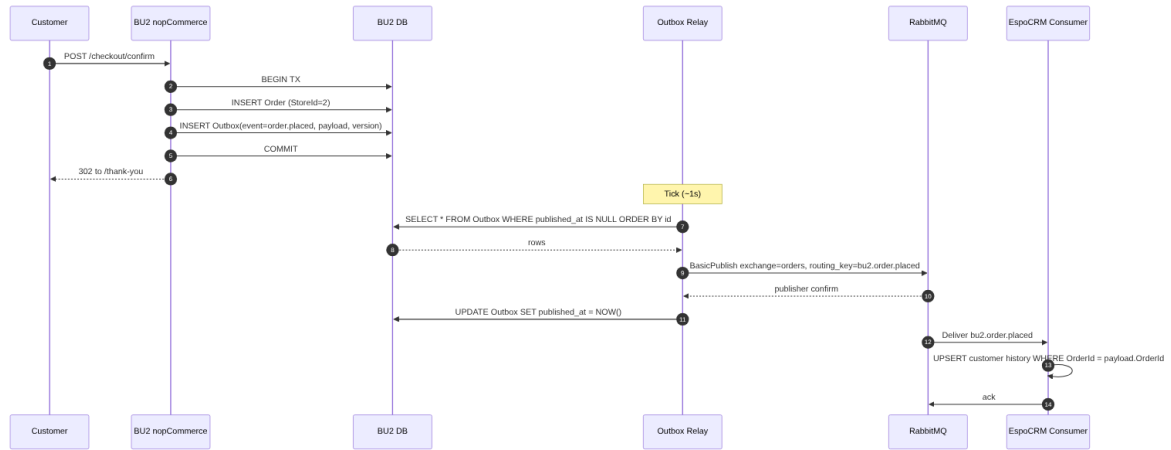


Figure 6: Flow 3: Transactional outbox plus idempotent consumer.

Flow 4 - ERP failure with circuit breaker (QA1, ADR-004)

BU2's local ERP becomes unreachable. The circuit breaker observes consecutive timeouts, opens, and short-circuits subsequent calls. The ACL substitutes a cached stock snapshot (TTL 5 min). The customer sees a "stock to be confirmed at fulfilment" notice. Recovery is via a half-open probe: a single test call is allowed; on success the breaker closes and traffic resumes hitting the ERP. Decision threshold: 5 consecutive failures or p95 latency above 2s in a 30s window (Polly defaults, tuned per ADR-004).

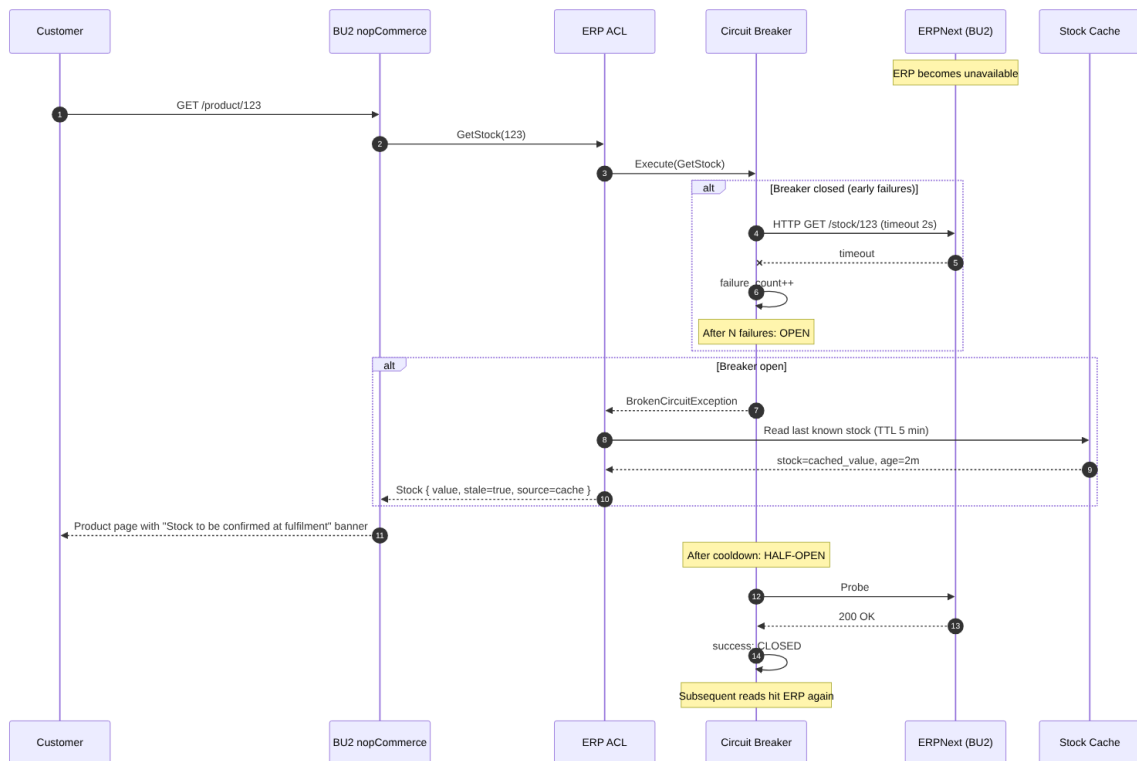


Figure 7: Flow 4: Circuit-breaker fallback to cached stock during ERP outage.

7. Architectural Decisions

This section records the five major architectural decisions in Michael Nygard’s ADR format (Status, Context, Decision, Consequences, Rejected Alternatives), as required by the brief. Each ADR traces back to its driver, pressure point, bounded context, and ADD iteration. All five are currently in *Proposed* status; status will move to *Accepted* as Part 2 implementation validates them.

7.1 ADR-001: Per-BU process and database isolation

Drivers: QA1 (primary), QA2 (supports). Pressure points: P1, P6, P7. Bounded contexts: BC2, BC3. ADD iteration: 1.

Status: Proposed.

Context: nopCommerce ships as a single deployable backed by one relational database, so a memory leak, slow plug-in, runaway query, or background-task failure in one part of the system degrades every other part of it. The platform’s multi-store feature keeps catalog rows separate when correctly configured, but it does not isolate process state, the DI container, the garbage collector, or the database connection pool, and the IgnoreStore-Limitations toggle (P6) sits one click away from collapsing every catalog isolation rule across the group. Three of the seven pressure points are direct artifacts of one-instance, one-database multi-tenancy: the global `Product.Price` field (P1), the global-bypass toggle (P6), and the role-only-not-store ACL dimension (P7). QA1 demands that a failure inside one BU cannot reach another BU; QA2 demands that one BU can re-price two hundred products without touching a setting another BU consumes. Both pressures point at the same answer: the two BUs cannot share a process or a database.

Decision: Each BU runs as its own ASP.NET Core 9 process with its own PostgreSQL database. The two instances share no process state, no DI container, no GC, no connection pool, and no relational database, so any failure inside one BU is structurally contained to that BU. The plug-in surface, the multi-store feature, and the rest of nopCommerce continue to work unchanged inside each instance: the multi-store concept simply stops being load-bearing for cross-BU isolation, because there is no longer such a thing as “the other BU inside this instance”. The shared substrate (Keycloak for identity, RabbitMQ for cross-BU events, Meiliseach for federated search) sits outside both BUs and is reached over the network, never through shared in-process state.

Consequences: QA1’s fault isolation is satisfied by construction since the BUs do not share a failure domain at the process or database level, and ADR-004’s circuit breaker on the per-BU ERP integration becomes the second layer of fault isolation rather than the only one. QA2’s pricing autonomy follows trivially because each BU has its own `Product` table with its own `Price` column, so a BU1 update writes to BU1’s database alone and BU2’s queries never see it; P1 dissolves at the structural level. P6 stops being a federation-wide hazard because the toggle, even if flipped, only affects the one BU it lives in; P7 (role-scoped-only ACL) becomes a within-BU concern with no group-wide leakage potential. The cost is operational doubling: two deployments to maintain, two database lifecycles (migrations, backups, monitoring), and a group-level “publish a product to every BU” workflow becomes an explicit multi-step admin operation rather than a single click

on a shared store. Identity becomes the only piece of cross-BU shared state that must remain in lock-step, which is precisely what ADR-002 is for.

Rejected alternatives: Hosting both BUs as two stores inside a single nopCommerce instance and relying on StoreMapping together with mandatory enforcement of the IgnoreStoreLimitations toggle was the prior proposal and was rejected because it satisfies QA2 in principle but not QA1 in practice: a single instance is a single failure domain, so a memory leak or a slow plug-in in one BU still degrades the other regardless of how thoroughly the catalog is scoped. A shared database with per-BU schemas was rejected on the same grounds plus the no-shared-database rule of the brief, since the database becomes the new single failure domain and the silent-collapse risk reappears one layer down. Per-BU containers fronting a shared database has the same defect: the database, not the process, is now the single point of failure, with the additional drawback that schema migrations cross BU boundaries. Adding a store identifier directly to the Product entity was a cleaner pure-schema option for the within-instance case but rippled through migrations, the admin UI, and every plug-in for no benefit once the instances are split. Extracting a separate Price or Catalog service was deferred because it is a major architectural move (data migration, broken admin paths, bidirectional sync) and the per-instance split already satisfies QA2 with no schema change.

7.2 ADR-002: Federate identity via Keycloak as external IdP

Driver: QA3 (supports QA1). Pressure point: P2. Bounded context: BC1. ADD iteration: 2.

Status: Proposed.

Context: nopCommerce currently owns customer identity, but its identity model is the wrong shape for a federated group: customer email and username are global with no DB-level uniqueness, customer roles have no store dimension, and there is no mechanism for one customer to be recognised across two BUs without re-registering. With per-BU databases now enforced (ADR-001), identity is the one piece of cross-BU shared state that must stay in lock-step, so a within-instance solution cannot work. The existing IExternalAuthenticationMethod plug-in slot offers a clean integration point that does not require touching the core, which is what makes the decision tractable rather than speculative.

Decision: The group therefore adopts Keycloak as its external IdP and turns each nopCommerce BU instance into an OIDC relying party through a custom Keycloak plugin implementing IExternalAuthenticationMethod. Group-level roles travel inside JWT claims while BU-local roles continue to live inside each nopCommerce instance, mapped from token claims on first login per BU. The authentication flow is Authorization Code with PKCE, with a five-minute access token, a thirty-minute refresh window, and a sliding session that keeps the user signed in across BUs.

Consequences: This satisfies QA3 directly and removes identity from the per-BU concern list entirely, with the side benefit that the email-uniqueness gap simply becomes Keycloak's problem and existing customer records survive as local profiles linked by Keycloak subject. The price is that identity becomes a group-level dependency: when Keycloak is

down, no BU can authenticate, which is the reason the production deployment runs an HA pair behind short-lived tokens and cookie-backed sessions. Existing customers must be migrated by email match on first SSO login, and the customer-facing login UX shifts from a local form to a redirect.

Rejected alternatives: The most dangerous alternative considered was to keep nopCommerce as the identity owner and replicate customers between BUs through events; that path was rejected because identity is the one domain where eventual consistency turns latency into a correctness bug, since a customer who cannot log in because their replication event has not yet propagated is locked out rather than merely slowed down. Once event-based identity is off the table, the remaining choices are which OIDC provider and which protocol to use. Hosted offerings such as Azure AD B2C, Auth0, or Cognito match Keycloak architecturally, but vendor lock-in and recurring cost are unwarranted when the integration seam is identical for any OIDC IdP and a future swap remains cheap. A bespoke group-level identity service would amount to a from-scratch rebuild of OIDC and was discarded on cost alone, while SAML was the only credible protocol alternative and was set aside because its XML payloads sit awkwardly with SPA and future mobile flows, leaving OIDC plus JWT as the cleaner long-term protocol.

7.3 ADR-003: Bridge `IEventPublisher` to RabbitMQ for cross-BU events

Driver: QA5 (supports QA4). Pressure point: P4. Bounded context: BC5. ADD iteration: 3.

Status: Proposed.

Context: nopCommerce already has an in-process event system through `IEventPublisher` together with auto-fired entity events from the repository layer, but it is in-process only, sequential, and non-durable, none of which satisfies QA5's requirement that order events reach BC4 reliably and survive consumer outages of at least twenty-four hours. With per-BU databases (ADR-001) the in-process publisher cannot reach across BUs at all, so a durable transport is no longer optional. The existing publisher is the right seam to extend but the wrong transport on its own.

Decision: The platform therefore bridges `IEventPublisher` to RabbitMQ through a transport-layer publisher and a transactional outbox so that correctness across the nopCommerce database and the broker holds without needing distributed transactions. Each order is persisted alongside a row in the outbox table inside the same database transaction, and a background relay later reads unpublished rows in order, publishes them to RabbitMQ, and marks them as published. The broker topology uses a topic exchange per concern with routing keys that carry the originating BU, and consumers run on durable queues with manual ack and dead-letter on parse or processing error after a small retry budget, with idempotency keyed on aggregate identity and version.

Consequences: The headline benefit is durability: orders survive consumer outages of any practical duration, every existing `IEventPublisher` call site stays untouched, and the dual-write hazard is removed by construction rather than by discipline. The same broker investment also pays for ADR-005's federated search at no extra operational cost, which is part of why iteration 3 lands before iteration 5 in the framework. The price is

a sub-second consumer-side latency from the two-step publish, an additional group-level component to run, and an outbox table that needs periodic pruning.

Rejected alternatives: The simplest alternative considered, a synchronous HTTP call from each BU to the CRM on order placement, was rejected because it would couple order placement to CRM availability and turn a CRM outage into either failed orders or silently dropped updates, both of which break QA5 directly. Kafka was the strongest second-place broker but its strengths sit in high-throughput streaming and long-window replay, neither of which match transactional commerce volumes; RabbitMQ’s per-message ack and dead-letter exchange map far more cleanly onto the “process each order exactly once with retries” shape at lower operational cost, with the option to swap if traffic patterns ever change. A polling integration in which the CRM scrapes each BU’s database was discarded as a clear violation of the no-shared-database rule and a leak of BU schema into the CRM, while reusing `IEventPublisher` across processes is not even technically possible because it resolves consumers through in-process DI and has no transport. Finally, publishing to RabbitMQ directly without an outbox simply reintroduces the dual-write problem the outbox was chosen to remove, since the database commit and the broker publish are not atomic and a crash between them either loses or ghosts the event.

7.4 ADR-004: BU-local ERP behind anti-corruption layer with circuit breaker

Driver: QA1. Pressure point: P3. Bounded contexts: BC2, BC3. ADD iteration: 4.

Status: Proposed.

Context: nopCommerce’s inventory model is warehouse-centric and carries no store dimension on the warehouse itself. ADR-001 already isolates BU failure domains at the process and database level, but inventory at checkout is a synchronous read against an external ERP, so the ERP integration becomes the second potential entry point for cross-BU fault propagation if it is centralised or shared. Inventory at checkout cannot be made fully asynchronous regardless of how stock updates are propagated, since the storefront has to know whether a SKU is available before it can confirm an order, so the read path needs a fast available source even when the ERP is degraded.

Decision: Each BU therefore runs its own local ERP, and the nopCommerce instance for that BU integrates with its ERP through a dedicated Anti-Corruption Layer with a Polly circuit breaker in front. The ACL translates ERP-specific stock and warehouse models into nopCommerce’s inventory abstraction so the rest of the platform never sees ERP-specific concepts, and the breaker trips from closed to open after a small number of consecutive failures or after sustained latency above threshold, then probes half-open after a cooldown. When the breaker is open the BU stays available in a degraded mode: browse and add-to-cart serve a cached stock snapshot with a default five-minute TTL, checkout discloses that stock will be confirmed at fulfilment, and confirmed orders enter a pending-fulfilment state that is reconciled once the ERP recovers.

Consequences: The arrangement satisfies QA1’s second layer of fault isolation: ADR-001 contains process and database failures, and this ADR contains ERP-integration failures so that a single ERP outage no longer escalates beyond its own BU. BU autonomy is preserved end-to-end since each ERP keeps its own database with no boundary-crossing

schema. Failure becomes observable rather than silent, which matters as much as the fault isolation itself. The honest cost is that stock figures during a degraded period can be wrong within bounded windows that are disclosed in the UI, a small fraction of confirmed orders may turn out to be unfulfillable after ERP recovery and require an explicit reconciliation policy, and the group inherits the operational burden of two ERP stacks rather than one.

Rejected alternatives: The most obvious alternative considered, a single shared ERP for the group, simply reproduces the original monolith problem one layer down: one ERP outage still becomes one group-wide outage, just at the inventory layer instead of the storefront. A synchronous ERP call on every page load with no cache makes the storefront as available as the slowest ERP in the group, which is indistinguishable from no fault isolation at all. An async-only ERP integration is appealing for write paths but breaks on the read path, since inventory at checkout is inherently synchronous: events can flow into nopCommerce, but the price-and-availability read still needs a fast available source. Replacing the breaker with a plain timeout was considered and rejected because a timeout still hits the slow ERP on every request, whereas the breaker stops calling entirely once the ERP is broken, which is the actual fault-isolation behaviour. The bulkhead pattern was not so much rejected as kept in addition: bulkheads protect connection pools per BU and the breaker protects the user from ongoing ERP failure, so the two complement each other rather than substitute for one another.

7.5 ADR-005: Federated Meilisearch with DB fallback strategy

Driver: QA4. *Pressure points:* P4 (cross-BU coordination), P1 (cross-BU discovery). *Bounded context:* BC5. *ADD iteration:* 5.

Status: Proposed.

Context: Cross-BU product discovery has no native solution in nopCommerce because each BU instance owns its own catalog database (ADR-001), so a federated search index that lives outside both BUs is required if a customer is to discover products across the group. QA4 also imposes a graceful-degradation requirement on top of this: search has to remain useful when the index itself is unavailable, otherwise an outage in shared infrastructure cascades back into every BU's storefront.

Decision: The group therefore adopts Meilisearch as its federated search index, populated asynchronously from each BU through ADR-003's outbox and broker so that no synchronous fan-out is needed at write time. Storefront search runs through a strategy pattern with a circuit breaker in which Meilisearch is the primary engine and BU-local database search is the fallback, with each indexed document tagged by its originating BU so that cross-BU search is the default and the UI can filter down to a single BU on demand. The read path issues each Meilisearch query with a one-and-a-half-second timeout; on timeout or error the breaker opens, the BU switches to local database search (which by design returns BU-scoped rather than group-scoped results), and the UI displays a non-blocking notice. Recovery is automatic: the breaker probes half-open at thirty-second intervals with a single request, and on success the BU resumes Meilisearch and drains its event backlog from RabbitMQ without manual intervention.

Consequences: This satisfies QA4 directly and turns cross-BU discovery into a real capability rather than a UI illusion, with the strategy pattern keeping the fallback path testable in isolation and the broker investment from ADR-003 reused at no extra cost. The trade is that fallback results during an index outage are BU-scoped rather than group-scoped, which is functionally degraded but still useful to the customer and is disclosed in the UI; index drift is bounded by RabbitMQ’s retention window and reconciles on reconnect, so eventual consistency holds without manual intervention.

Rejected alternatives: Elasticsearch would satisfy the same architectural shape but at a significantly heavier operational cost (JVM tuning, multi-node baseline, more knobs to misconfigure), whereas Meilisearch covers the storefront search use case at a fraction of the operational footprint and the strategy pattern leaves the door open to swap engines later if relevance or scale ever demand it. Database full-text search alone would either force a shared catalog database, which contradicts BU autonomy, or a synchronous fan-out per BU, which compounds latency and creates the direct cross-BU coupling the federation is supposed to avoid. A hosted SaaS option such as Algolia was set aside on the same grounds as the hosted IdPs in ADR-002: vendor lock-in, recurring cost, and data-residency questions that are not justified in this context, with the plug point being the actual decision and the engine remaining replaceable. Pushing the failure mode onto the customer with a plain “search unavailable” message was rejected as fail-closed behaviour that violates QA4 and erodes user trust without any architectural benefit.

8. Risk and Validation Plan

This section satisfies the brief deliverable “risk and validation plan: main architectural risks; what you will build or measure to de-risk them”.

8.1 Risk Register

Risks are scored on a 1–3 scale for probability (P) and impact (I), with score = $P \times I$ (1–9). Risks with score ≥ 6 are treated as primary risks for Part 2 mitigation work. Each risk traces back to one or more ADRs.

Table 4: Risk register

ID	Risk	P	I	Score	Trace
R1	Keycloak becomes a group-wide login SPOF	1	3	3	ADR-002
R2	Outbox/idempotency bug causes duplicate or lost cross-BU order events	2	3	6	ADR-003, QA5
R3	BU deployments drift on nopCommerce version, plug-in roster, or security-patch cadence over time	3	2	6	ADR-001
R4	Customer migration to Keycloak loses or duplicates accounts at cutover	2	3	6	ADR-002
R5	RabbitMQ outage stalls the cross-BU pipeline	1	2	2	ADR-003
R7	BU ERP circuit-breaker tuning produces flapping (open / closed loop)	2	2	4	ADR-004
R8	Pending-fulfilment orders during ERP outage cannot be honoured at recovery	1	2	2	ADR-004
R9	Two BU teams diverge on product/category schema, breaking federated search	3	2	6	ADR-001, ADR-005
R10	Token-claim to BU-role mapping drifts between BUs, causing role bleed	1	3	3	ADR-002, QA3

8.2 Risk Heatmap

Risks plotted on a 3×3 probability $\times$ impact grid (Table 5). Cells are shaded by score, with bold IDs marking primary risks (score ≥ 6). The register shows residual risk after the planned architectural mitigations are in place: structural mitigations (Keycloak HA, single-source-of-truth role mapping, durable queues) push some probabilities to Low, while drift risks whose mitigation depends on active management remain at Medium or High probability.

Table 5: Risk heatmap (probability $\times$ impact)

	I=1 (Low)	I=2 (Medium)	I=3 (High)
P=3 (High)		R3, R9	
P=2 (Medium)		R7	R2, R4
P=1 (Low)		R5, R8	R1, R10

Two patterns emerge. The high-impact column (R1, R2, R4, R10) holds the build-quality and external-dependency risks: R2 and R4 are medium probability and stay primary because they hinge on Part 2 implementation correctness (outbox, migration), while R1 and R10 drop to low probability because their mitigations are structural (Keycloak HA, config-as-code role mapping). The medium-impact column reveals the drift risks: R3 and R9 climb to high probability because their mitigations depend on active management of two independently maintained BU deployments rather than on a structural guarantee. R7 sits at medium probability as a build-time tuning concern. R5 and R8 drop to low probability because the architecture handles their failure modes by design (durable outbox; cached stock plus reconciliation). Primary targets for Part 2 mitigation work are therefore R2, R3, R4, and R9.

8.3 Mitigations

Table 6: Risk mitigations

ID	Mitigation
R1	Keycloak HA pair (primary + standby with shared DB); short-lived access tokens with refresh; storefront cookie session continues to work for already-logged-in users until token expiry.
R2	Transactional outbox (DB + outbox row in same TX); idempotent consumers keyed on (<code>aggregate_id</code> , <code>version</code>); integration test simulating relay crash mid-publish.
R3	Shared deployment templates (Docker Compose / Helm) and CI pipelines per BU; quarterly version-sync calendar; common security-advisory subscription; integration test that asserts both BUs run the same nopCommerce version unless an exception is recorded.
R4	Cutover plan: link by email on first post-migration login; admin reconciliation tool; dry-run on a copy of production data.
R5	Durable queues + persistent messages; outbox retains unpublished rows; relay backs off and resumes on reconnect; alerting on relay lag.
R7	Slow-start half-open with single probe; metrics dashboard for breaker state transitions; alert on flap rate.
R8	Reconciliation policy: customer notified, refund or backorder offered; bounded by ERP cache TTL (≤ 5 min).
R9	Indexer ACL normalises BU schemas into a single search document shape; integration test per BU mapping.
R10	Single source of truth in Keycloak for the claim-to-role-prefix mapping table; checked into git; loaded at BU startup.

9. Evolution Roadmap

The architecture is reached not in a single jump but as an evolution from a baseline. Iteration 0 is nopCommerce as it ships today; each subsequent iteration adds one focused capability driven by a single QA scenario, as detailed in Section 6. The order below matches the dependency between iterations: per-BU process and database isolation comes first because everything afterwards assumes it, identity comes second because every later iteration assumes one sign-on identity, and the broker (Iteration 3) lands before federated search (Iteration 5) because the search re-index pipeline reuses it.

Iteration 0: Current state. The starting point is nopCommerce as a single deployable backed by one relational database, with the seven pressure points P1-P7 identified in Section 2. The external systems the target architecture will integrate with (Keycloak, ERPNext, Odoo, RabbitMQ, Meilisearch, EspoCRM) exist in the wider ecosystem but are not connected to the platform.

Iteration 1 (QA1+QA2, ADR-001): Per-BU process and database isolation. The single nopCommerce deployable is split into one ASP.NET Core process per BU, each backed by its own PostgreSQL database. The two BUs share no process state, no DI container, and no relational database. The multi-store feature continues to work inside each instance but is no longer load-bearing for cross-BU isolation.

Iteration 2 (QA3, ADR-002): Federated identity. Keycloak is added as the group identity provider and each nopCommerce instance becomes an OIDC relying party through a custom plug-in implementing `IExternalAuthenticationMethod`. Group claims travel inside the JWT and are mapped to BU-prefixed local roles on first login per BU.

Iteration 3 (QA5, ADR-003): Cross-BU async coordination. RabbitMQ is added as the durable transport. An outbox table inside each BU's database holds the to-be-published events and is drained by a relay; EspoCRM consumes order events idempotently into the cross-BU customer profile. The in-process `IEventPublisher` keeps working for in-process consumers; the broker handles only events that need to cross processes.

Iteration 4 (QA1, ADR-004): Per-BU ERP behind ACL and breaker. Each BU stands up its own local ERP behind an Anti-Corruption Layer with a circuit breaker. When the ERP is unreachable, a cached stock snapshot serves the open-circuit fallback so browse and add-to-cart continue with stale stock and checkout discloses "stock to be confirmed at fulfilment". This is the second layer of fault isolation, complementing the process and database split from Iteration 1.

Iteration 5 (QA4, ADR-005): Federated search with fallback. Meilisearch is added as the federated index, populated asynchronously via the broker stream from Iteration 3. Storefront search uses a strategy with circuit breaker, falling back to the BU's own database (BU-scoped results) when the index is unhealthy. The database search path is permanent: it is the fallback, not a transitional artifact.